120226

February 21, 2026

0.0.1 Imports

[3]:
```
## Nessacery imports for entire lab done here for easy management
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt
import hyperspy.api as hs
from scipy import optimize
```

0.0.2 Functions

[59]:
```
## Defining perodic function with a sine and cosine term, variables allow␣
↪manipulation of all characteristics,
## while the presence of a sin and cos allow variation of amplitude etc in a␣
↪period
def perodic(x,a,b,c,d,e,f,g):
    return (a*np.sin(b*np.deg2rad(x) + c) + (d*np.cos(e*np.deg2rad(x) + f)) + g)


## Definig perodic plot function to fit the perodic function to one slice of␣
↪data
def periodic_fit_plot(x_variable, y_variable,initial, plot = False):
## Using optimize.curve_fit to fit a perodic funtion to our x and y data,␣
↪intial values for parameters taken from input
    params_sin, params_sin_covariance = optimize.
↪curve_fit(perodic,x_variable,y_variable,p0=initial,maxfev=30000)

## Finding estimated uncertainties for each parameter
    errs = np.sqrt(np.diag(params_sin_covariance))

## creating a dummy variable for plotting purposes using linspace, parameters␣
↪of linspace ensure it varies from lowest to highest x value regarless of␣
↪order in array
    dummy = np.linspace(np.min(x_variable),np.max(x_variable),360)

## Calculating residuals
    difference = perodic(x_variable,*params_sin) - y_variable
```

```
## Calculating r squared for goodness of fit

## Squared sum of residuals
    ss_res = np.sum(difference**2)

## Squared sum of data - mean
    ss_tot = np.sum((y_variable- np.mean(y_variable))**2)

## Calculate r_squared per equation
    r_squared = 1 - (ss_res/ss_tot)

    ## Rounding frequencies, ensuring single values at 0/360
    params_sin[1] = np.round(params_sin[1], decimals=0)
    params_sin[4] = np.round(params_sin[4], decimals=0)
    if plot == True:
## Setting up subplots
        plt.subplots(2, 1, gridspec_kw={'height_ratios':␣
 ↪[2,1]},sharex=True,figsize=[10,6]) # height ratios make sizes right, sharex␣
 ↪makes plots share x axis
## Subplot 2 shows residuals
        plt.subplot(212)
        plt.plot(x_variable,difference,'or')
        plt.xlabel("Current (A)")
        plt.ylabel("Residuals")

        plt.grid()
        plt.tight_layout()
## Subplot 1 with fit and raw data
        plt.subplot(211)
        plt.scatter(x_variable, y_variable,label="Raw Data")
        plt.plot(dummy,perodic(dummy,*params_sin),color="r",label="Fitted␣
 ↪Function") ## using curvefit parameters and linear function to plot function
        plt.
 ↪fill_between(dummy,perodic(dummy,*(params_sin+errs)),perodic(dummy,*(params_sin-errs)),alph
 ↪2) ## highlights linear fit within 1 SD of both parameters, too small to see␣
 ↪for this data
        plt.plot([], [], ' ', label="R^2 = {0:.4f}".format(r_squared)) ##␣
 ↪displaying r squared value
        plt.grid()
        plt.legend()

    return(params_sin,errs)
```

[90]:
```
def decomp_data(path, components, algo):
    raw_data = hs.load(path)
    raw_data_end = np.load(path)
    data_hs_func = hs.signals.Signal1D(raw_data.data)
```

```
    data_hs_func.decomposition(algorithm=algo)
    sc = data_hs_func.get_decomposition_model(components)
    decomposed_data = sc.data

    return decomposed_data, raw_data_end

def decomp_and_fit(path, components, algo):
    decomp = decomp_data(path,components,algo)

    decomp_params =␣
 ↪periodic_fit_plot(x,decomp[0][0][20],[20,1,1,20,1,1,5],plot=False)
    params = periodic_fit_plot(x,decomp[1][0][20],[20,1,1,20,1,1,5],plot=False)

    return decomp[0], decomp[1], decomp_params, params
```

0.0.3 Data Processing

[97]:
```
## Using the smaller initial data set to investigate and test the decomp␣
 ↪functions
# Using just SVD at the moment but will test with other algorithms later
```

[63]:
```
raw_data = hs.load("binned_diff_20210421_160712_PT1D1.npy")
raw_data1 = np.load("binned_diff_20210421_160712_PT1D1.npy")
raw_data1.shape
```

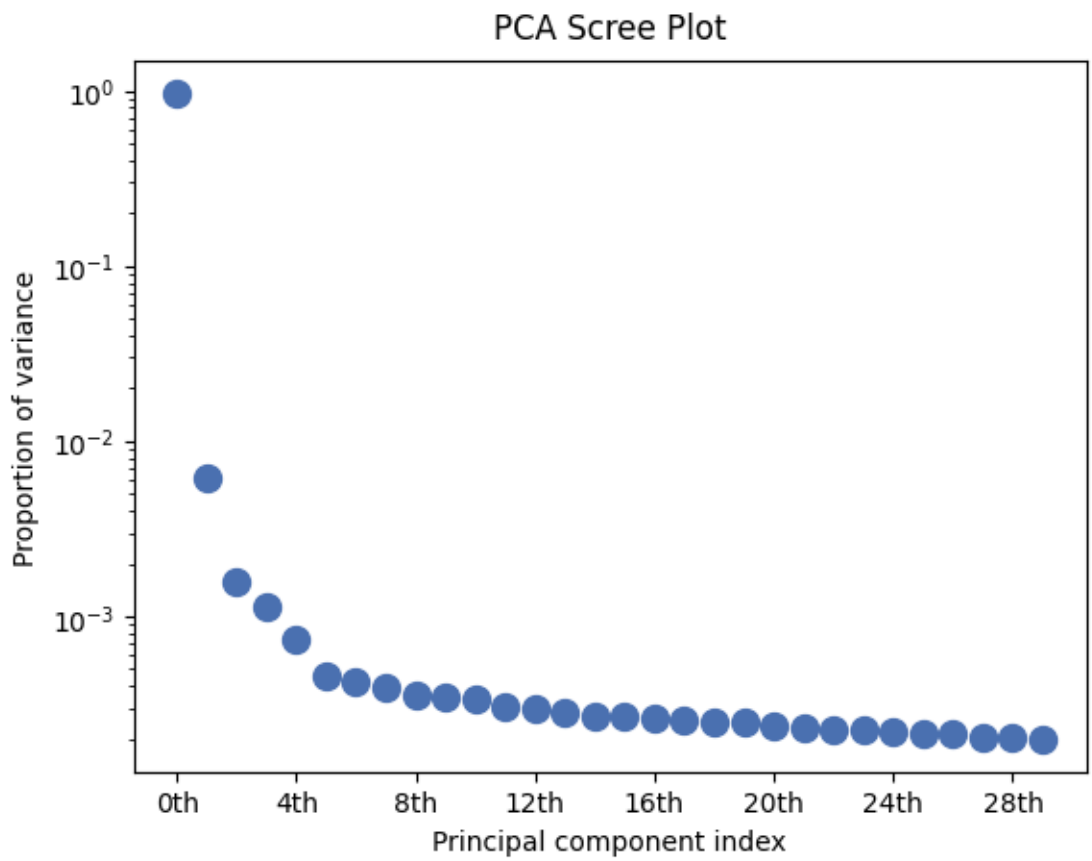
[63]: (35, 35, 360)

[ ]:
```
## Investingating Scree Plot
# Seems that the first 4 components hold significance, we will use those for␣
 ↪this test
```

[98]:
```
raw_data1 = hs.signals.Signal1D(raw_data.data)
raw_data1.decomposition()
raw_data1.plot_explained_variance_ratio()
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

[98]:
```
<Axes: title={'center': '\nPCA Scree Plot'}, xlabel='Principal component index',
ylabel='Proportion of variance'>
```

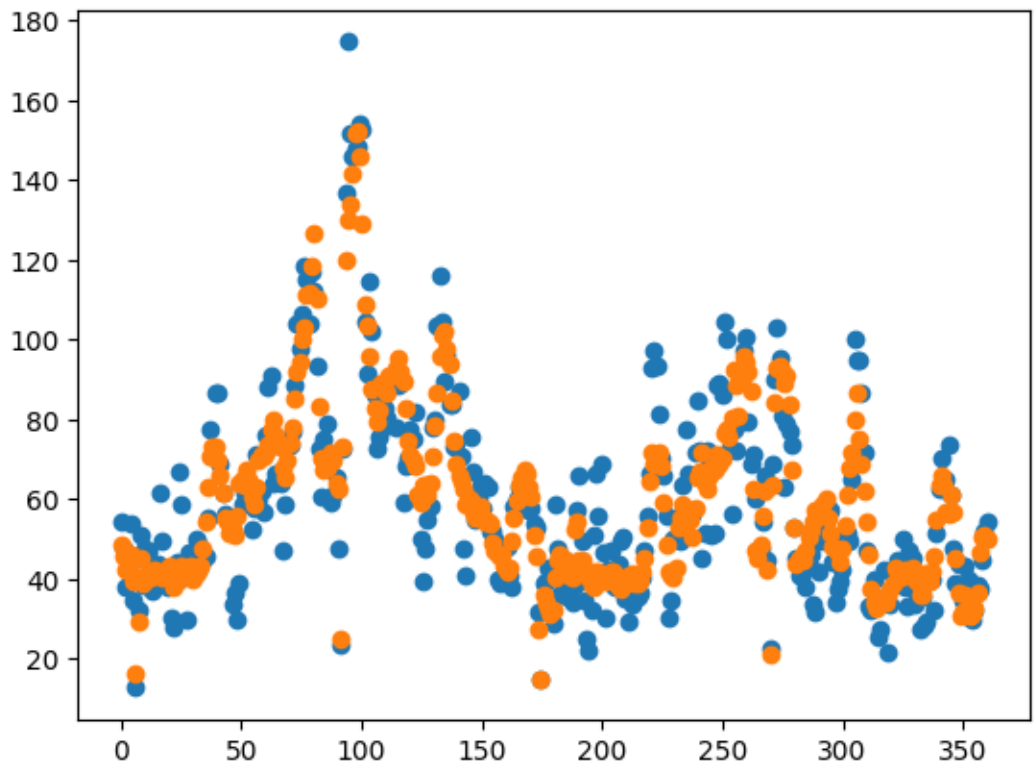


[99]:
```
## Using function to decompose data using SVD algo and 4 components
test = decomp_data("binned_diff_20210421_160712_PT1D1.npy",4,"SVD")
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

[105]:
```
## Plotting the raw and decomped data from selected pixel

decomp, raw = test

x= np.linspace(0,360,360)

plt.scatter(x, raw[25][25])
plt.scatter(x, decomp[25][25])
```

[105]:
```
<matplotlib.collections.PathCollection at 0x7f1528ab7670>
```



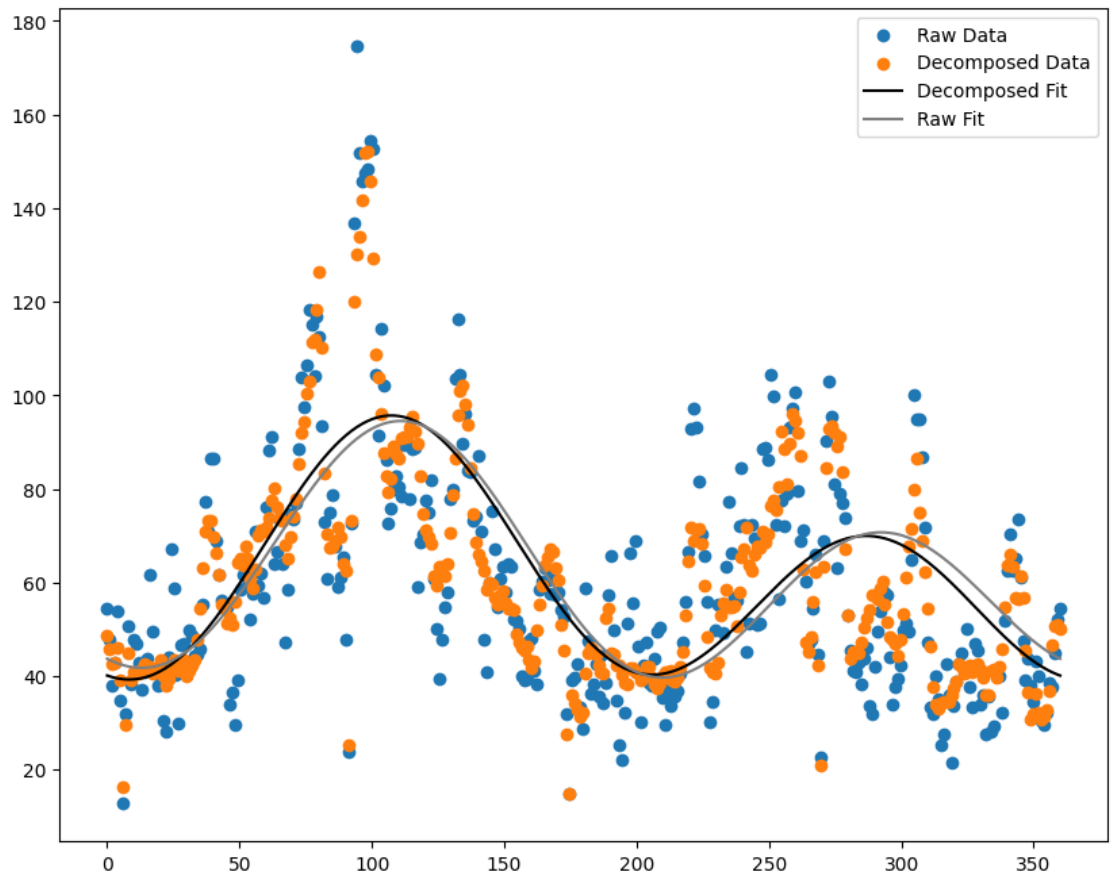
[106]:
```
## Plotting using functions for both raw and decomped data sets

decomp_params = periodic_fit_plot(x,decomp[25][25],[20,1,1,20,1,1,5],plot=False)
params = periodic_fit_plot(x,raw[25][25],[20,1,1,20,1,1,5],plot=False)
```

[108]:
```
## Ensuring that we have integer frequencies, i.e single valued at 0/360 (it is)

print(params[0])
```

```
[ 11.94570199  -1.          3.42869217 -20.51694809   2.
  -0.7499646   62.08681172]
```

[109]:
```
## Plotting raw and decomped data and fits
plt.figure(figsize=[10,8])
plt.scatter(x, raw[25][25],label="Raw Data")
plt.scatter(x, decomp[25][25],label="Decomposed Data")

plt.plot(x,perodic(x,*decomp_params[0]),color="black",label="Decomposed Fit")
plt.plot(x,perodic(x,*params[0]),color="grey",label="Raw Fit")
plt.legend()
```

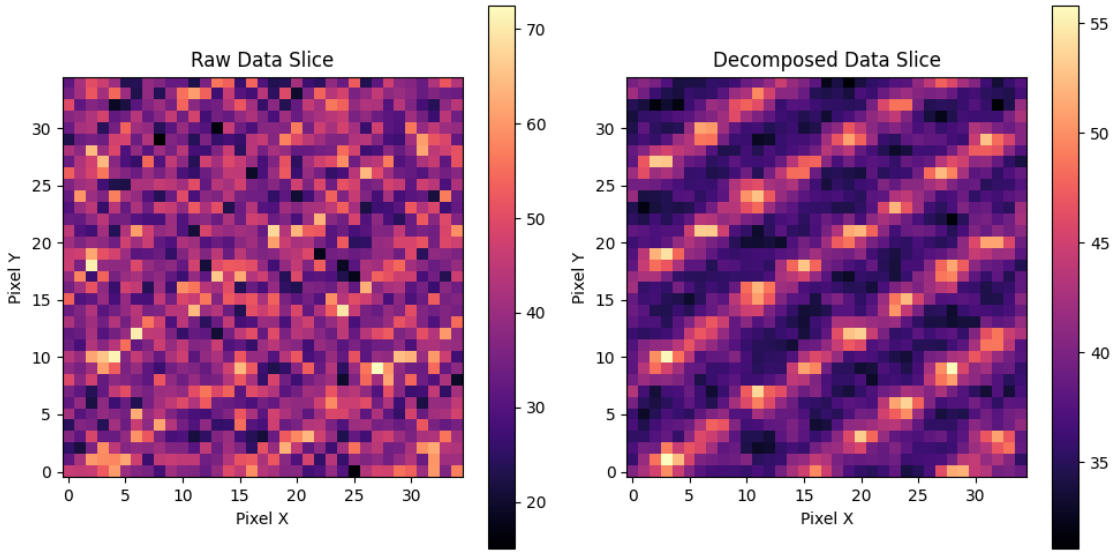
[109]:
```
<matplotlib.legend.Legend at 0x7f1528a19e20>
```



[110]:
```
## PLot above shows that fit does change when decomping data. This is expected.␣
↪not yet assesed goodness of fit, can do that later to quantify if / how much␣
↪decomping and denoising is helping
```

[104]:
```
## Using imshow to visualise slice of both raw and decomposed datasets

slice_index = 10  # azimuthal slice to visualize

fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Ground truth
im0 = axes[0].imshow(raw[:, :, slice_index], cmap='magma', origin='lower')
axes[0].set_title('Raw Data Slice')
plt.colorbar(im0, ax=axes[0])

# Noisy input
im1 = axes[1].imshow(decomp[:, :, slice_index], cmap='magma', origin='lower')
axes[1].set_title('Decomposed Data Slice')
plt.colorbar(im1, ax=axes[1])
```

```
# Common labels
for ax in axes:
    ax.set_xlabel('Pixel X')
    ax.set_ylabel('Pixel Y')

plt.tight_layout()
plt.show()
```



[111]:
```
## Above shows huge inprovement in visual clarity of the scans after␣
↪decomposing with SVD compared to raw data.
```